Southern New Hampshire University

Project One

Adalberto Flores

Southern New Hampshire University

CS-300

Michael Susalla

October 11, 2026

Southern New Hampshire University

Milestone One: Vector Data Structures

```
// Course Object Definition
Structure Course:
    courseNumber : String
    title : String
    prerequisites : Vector of String
End Structure


// Method 1: Load Courses & Validate Prerequisites
Method loadCourses(filename : String) Returns Vector of Course:
    Initialize courses as Vector of Course
    Initialize validCourseIds as Vector of String
    Initialize rawFileLines as Vector of String

    Open file with filename for reading
    If file is not open Then
        Output "Error: Unable to open file: " + filename
        Return courses
    End If

    // Pass 1: Syntax check and collect valid course numbers
    While file has another line:
        Read line from file
        Split line by comma into tokens


        If number of tokens < 2 Then
```

```
        Output "Error: Invalid course format (minimum 2 parameters required): " + line
        Continue to next line
    End If

    Add tokens[0] to validCourseIds
    Add line to rawFileLines
End While
Close file

// Pass 2: Verify prerequisites and populate the vector
For each line in rawFileLines:
    Split line by comma into tokens
    Create Course newCourse
    Set newCourse.courseNumber = tokens[0]
    Set newCourse.title = tokens[1]
    Set isValid = true

    For index = 2 to (number of tokens - 1):
        Set prereq = tokens[index]

        // Verify prerequisite exists in the loaded course IDs
        Set prereqFound = false
        For each validId in validCourseIds:
            If validId equals prereq Then
                Set prereqFound = true
                Exit inner loop
            End If
```

```
            End For


            If prereqFound is false Then
                Output "Error: Prerequisite " + prereq + " does not exist in file for course " +
newCourse.courseNumber
                Set isValid = false
            Else
                Add prereq to newCourse.prerequisites
            End If
        End For


        // Only add courses that have completely valid prerequisites
        If isValid is true Then
            Add newCourse to courses
        Else
            Output "Skipping course " + newCourse.courseNumber + " due to missing
prerequisites."
        End If
    End For


    Output "Courses successfully loaded into Vector."
    Return courses
End Method




// Method 2: Search for a Single Course
Method searchCourse(courses : Vector of Course, targetCourseNumber : String):
    For each course in courses:
```

```
        If course.courseNumber equals targetCourseNumber Then
            Output course.courseNumber + ", " + course.title

            If course.prerequisites is empty Then
                Output "Prerequisites: None"
            Else
                Output "Prerequisites: "
                For each prereq in course.prerequisites:
                    Output prereq
                End For
            End If
            Return
        End If
    End For

    Output "Course " + targetCourseNumber + " not found."
End Method


// Method 3: Display All Courses in Alphanumeric Order
Method printCourses(courses : Vector of Course):
    // Sort courses in alphanumeric order by courseNumber (e.g., QuickSort or
SelectionSort)
    Sort courses ascending by course.courseNumber

    For each course in courses:
        Output course.courseNumber + ", " + course.title
    End For
```

```
End Method



// Main Menu Program Routine
Method main():
    Initialize courses as Vector of Course
    Set isLoaded = false
    Set choice = 0

    While choice != 9:
        Output "Menu:"
        Output "  1. Load Courses"
        Output "  2. Display Courses"
        Output "  3. Find Course"
        Output "  9. Exit"
        Output "What would you like to do? "
        Read choice from user

        If choice == 1 Then
            Output "Enter the data file name: "
            Read filename from user
            Set courses = loadCourses(filename)
            If courses is not empty Then
                Set isLoaded = true
            End If

        Else If choice == 2 Then
```

```
        If isLoaded is true Then
            Call printCourses(courses)
        Else
            Output "Please load the course data first."
        End If

    Else If choice == 3 Then
        If isLoaded is true Then
            Output "What course do you want to know about? "
            Read targetCourse from user
            Convert targetCourse to uppercase
            Call searchCourse(courses, targetCourse)
        Else
            Output "Please load the course data first."
        End If

    Else If choice == 9 Then
        Output "Thank you for using the course planner!"

    Else
        Output choice + " is not a valid option."
    End If
  End While
End Method
```

Southern New Hampshire University

Milestone Two: Hash Table Data Structures

```
// Course Object Definition
Structure Course:
    courseNumber : String
    title : String
    prerequisites : List of String
End Structure

Constant DEFAULT_SIZE = 179

// Hash Table Node for Separate Chaining
Structure Node:
    course : Course
    key : Unsigned Integer
    next : Pointer to Node
End Structure

Structure HashTable:
    nodes : Array of Node
    tableSize : Unsigned Integer
End Structure


// Method 1: Hash Function
Method hash(courseNumber : String, tableSize : Integer) Returns Unsigned Integer:
    hashValue = 0
    For each character in courseNumber do:
```

```
        hashValue = hashValue + ASCII value of character
    End For
    Return hashValue MODULO tableSize
End Method


// Method 2: Initialize Hash Table
Method initializeHashTable(size : Unsigned Integer) Returns HashTable:
    Initialize table as HashTable
    table.tableSize = size
    Resize table.nodes to size

    For index = 0 to size - 1 do:
        table.nodes[index].key = UINT_MAX
        table.nodes[index].next = null
    End For

    Return table
End Method


// Method 3: Insert Course into Hash Table
Method insertCourse(table : Pointer to HashTable, course : Course):
    bucket = hash(course.courseNumber, table.tableSize)
    head = address of table.nodes[bucket]

    // Empty bucket
```

```
    If head.key == UINT_MAX Then
        head.key = bucket
        head.course = course
        head.next = null
    Else
        // Collision: Traverse to tail of chain
        current = head
        While current.next != null do:
            current = current.next
        End While

        newNode = allocate Node
        newNode.key = bucket
        newNode.course = course
        newNode.next = null
        current.next = newNode
    End If
End Method



// Method 4: Search Hash Table
Method searchCourseHashTable(table : Pointer to HashTable, courseNumber : String)
Returns Course:
    Initialize emptyCourse as Course
    emptyCourse.courseNumber = ""
    emptyCourse.title = ""

    bucket = hash(courseNumber, table.tableSize)
```

```
    current = address of table.nodes[bucket]

    If current == null or current.key == UINT_MAX Then
        Return emptyCourse
    End If

    While current != null do:
        If current.course.courseNumber equals courseNumber Then
            Return current.course
        End If
        current = current.next
    End While

    Return emptyCourse
End Method


// Method 5: Load Courses & Validate Prerequisites
Method loadCourseHashTable(filename : String, table : Pointer to HashTable) Returns
Boolean:
    Open file with filename for reading
    If file is not open Then
        Output "Error: Unable to open file: " + filename
        Return false
    End If

    Initialize validCourseIds as List of String
    Initialize rawFileLines as List of String
```

```
// Pass 1: Syntax check and collect valid course IDs
While file has another line do:
    line = read next line from file
    tokens = split line by ","

    If length of tokens < 2 Then
        Output "Error: Invalid line format (minimum 2 parameters required): " + line
        Continue to next line
    End If

    Append tokens[0] to validCourseIds
    Append line to rawFileLines
End While
Close file

// Pass 2: Verify prerequisites and insert valid courses into Hash Table
For each line in rawFileLines do:
    tokens = split line by ","
    Initialize newCourse as Course
    newCourse.courseNumber = tokens[0]
    newCourse.title = tokens[1]
    isValid = true

    For index = 2 to (length of tokens - 1) do:
        prereq = tokens[index]
        prereqFound = false
```

```
            For each validId in validCourseIds do:
                If validId equals prereq Then
                    prereqFound = true
                    Exit inner loop
                End If
            End For

            If prereqFound is false Then
                Output "Error: Prerequisite " + prereq + " does not exist in file for course " +
newCourse.courseNumber
                isValid = false
            Else
                Append prereq to newCourse.prerequisites
            End If
        End For

        // Only insert if all prerequisites are valid; skip otherwise
        If isValid is true Then
            Call insertCourse(table, newCourse)
        Else
            Output "Skipping course " + newCourse.courseNumber + " due to missing
prerequisites."
        End If
    End For

    Output "Courses successfully loaded into Hash Table."
    Return true
```

```
End Method


// Method 6: Print Single Course Details
Method printCourseDetails(table : Pointer to HashTable, courseNumber : String):
    course = searchCourseHashTable(table, courseNumber)

    If course.courseNumber equals "" Then
        Output "Course " + courseNumber + " not found."
        Return
    End If

    Output course.courseNumber + ", " + course.title

    If length of course.prerequisites == 0 Then
        Output "Prerequisites: None"
    Else
        Output "Prerequisites:"
        For each prereq in course.prerequisites do:
            Output prereq
        End For
    End If
End Method


// Method 7: Display All Courses in Alphanumeric Order
Method displayAllCourses(table : Pointer to HashTable):
```

```
    Initialize allCourses as List of Course

    // Extract all courses from buckets into a temporary list
    For index = 0 to (table.tableSize - 1) do:
        If table.nodes[index].key != UINT_MAX Then
            Append table.nodes[index].course to allCourses
        End If

        current = table.nodes[index].next
        While current != null do:
            Append current.course to allCourses
            current = current.next
        End While
    End For

    // Sort the list alphanumerically
    Sort allCourses ascending by courseNumber

    For each course in allCourses do:
        Output course.courseNumber + ", " + course.title
    End For
End Method


// Main Menu Program Routine
Method main():
    courseTable = initializeHashTable(DEFAULT_SIZE)
```

```
isLoaded = false
choice = 0

While choice != 9 do:
    Output "Menu:"
    Output "  1. Load Courses"
    Output "  2. Display Courses"
    Output "  3. Find Course"
    Output "  9. Exit"
    Output "What would you like to do? "
    Read choice from user

    If choice == 1 Then
        Output "Enter the data file name: "
        Read filename from user
        isLoaded = loadCourseHashTable(filename, address of courseTable)

    Else If choice == 2 Then
        If isLoaded is true Then
            Call displayAllCourses(address of courseTable)
        Else
            Output "Please load the course data first."
        End If

    Else If choice == 3 Then
        If isLoaded is true Then
            Output "What course do you want to know about? "
```

```
            Read targetCourse from user
            Convert targetCourse to uppercase
            Call printCourseDetails(address of courseTable, targetCourse)
        Else
            Output "Please load the course data first."
        End If

    Else If choice == 9 Then
        Output "Thank you for using the course planner!"

    Else
        Output choice + " is not a valid option."
    End If
  End While
End Method
```

Southern New Hampshire University

Milestone Three: Banary Search Tree

```
// Course Object Definition
Structure Course:
    courseNumber : String
    title : String
    prerequisites : Vector of String
End Structure


// BST Node Definition
Structure Node:
    course : Course
    left : Pointer to Node
    right : Pointer to Node

    Node(aCourse : Course):
        Set course = aCourse
        Set left = null
        Set right = null
End Structure


// Binary Search Tree Structure
Structure BinarySearchTree:
    root : Pointer to Node

    Constructor():
```

```
        Set root = null
End Structure



// Method 1: Recursive Node Insertion
Method addNode(currentNode : Pointer to Node, aCourse : Course):
    If aCourse.courseNumber is less than currentNode.course.courseNumber Then
        If currentNode.left is null Then
            Set currentNode.left = new Node(aCourse)
        Else
            Call addNode(currentNode.left, aCourse)
        End If
    Else
        If currentNode.right is null Then
            Set currentNode.right = new Node(aCourse)
        Else
            Call addNode(currentNode.right, aCourse)
        End If
    End If
End Method



// Method 2: Insert Course into BST
Method insert(bst : Pointer to BinarySearchTree, aCourse : Course):
    If bst.root is null Then
        Set bst.root = new Node(aCourse)
    Else
```

```
        Call addNode(bst.root, aCourse)
    End If
End Method


// Method 3: In-Order Traversal (Prints Alphanumerically Sorted List)
Method inOrder(node : Pointer to Node):
    If node is not null Then
        Call inOrder(node.left)
        Output node.course.courseNumber + ", " + node.course.title
        Call inOrder(node.right)
    End If
End Method


// Method 4: Search Course in BST
Method search(bst : Pointer to BinarySearchTree, courseNumber : String) Returns
Course:
    Set currentNode = bst.root


    While currentNode is not null:
        If currentNode.course.courseNumber equals courseNumber Then
            Return currentNode.course
        Else If courseNumber is less than currentNode.course.courseNumber Then
            Set currentNode = currentNode.left
        Else
            Set currentNode = currentNode.right
        End If
```

```
    End While

    Initialize emptyCourse as Course
    Set emptyCourse.courseNumber = ""
    Set emptyCourse.title = ""
    Return emptyCourse
End Method


// Method 5: Load Courses & Validate Prerequisites
Method loadCourses(filename : String, bst : Pointer to BinarySearchTree) Returns
Boolean:
    Open file with filename for reading
    If file is not open Then
        Output "Error: File could not be opened: " + filename
        Return false
    End If

    Initialize validCourseIds as Vector of String
    Initialize rawFileLines as Vector of String

    // Pass 1: Syntax check and collect valid course IDs
    While file has another line:
        Read line from file
        Split line by comma into tokens

        If count of tokens < 2 Then
            Output "Error: Line does not have enough parameters: " + line
```

```
        Continue to next line
    End If

    Add tokens[0] to validCourseIds
    Add line to rawFileLines
End While
Close file

// Pass 2: Verify prerequisites and insert valid courses into the BST
For each line in rawFileLines:
    Split line by comma into tokens
    Create Course newCourse
    Set newCourse.courseNumber = tokens[0]
    Set newCourse.title = tokens[1]
    Set isValid = true

    For index = 2 to (count of tokens - 1):
        Set prereq = tokens[index]
        Set prereqFound = false

        For each validId in validCourseIds:
            If validId equals prereq Then
                Set prereqFound = true
                Exit inner loop
            End If
        End For
```

```
            If prereqFound is false Then
                Output "Error: Prerequisite " + prereq + " does not exist in file for course " + newCourse.courseNumber
                Set isValid = false
            Else
                Add prereq to newCourse.prerequisites
            End If
        End For

        // Only insert if all prerequisites exist; skip otherwise
        If isValid is true Then
            Call insert(bst, newCourse)
        Else
            Output "Skipping course " + newCourse.courseNumber + " due to missing prerequisites."
        End If
    End For

    Output "Courses successfully loaded into Binary Search Tree."
    Return true
End Method


// Method 6: Print Single Course Details
Method printCourseDetails(bst : Pointer to BinarySearchTree, courseNumber : String):
    Set course = search(bst, courseNumber)

    If course.courseNumber equals "" Then
```

```
        Output "Course " + courseNumber + " not found."
        Return
    End If

    Output course.courseNumber + ", " + course.title

    If course.prerequisites is empty Then
        Output "Prerequisites: None"
    Else
        Output "Prerequisites:"
        For each prereq in course.prerequisites:
            Output prereq
        End For
    End If
End Method


// Main Menu Program Routine
Method main():
    Create BinarySearchTree bst
    Set isLoaded = false
    Set choice = 0

    While choice != 9:
        Output "Menu:"
        Output "  1. Load Courses"
        Output "  2. Display Courses"
```

```
Output "  3. Find Course"
Output "  9. Exit"
Output "What would you like to do? "
Read choice from user

If choice == 1 Then
    Output "Enter the data file name: "
    Read filename from user
    Set isLoaded = loadCourses(filename, address of bst)

Else If choice == 2 Then
    If isLoaded is true Then
        Call inOrder(bst.root)
    Else
        Output "Please load the course data first."
    End If

Else If choice == 3 Then
    If isLoaded is true Then
        Output "What course do you want to know about? "
        Read targetCourse from user
        Convert targetCourse to uppercase
        Call printCourseDetails(address of bst, targetCourse)
    Else
        Output "Please load the course data first."
    End If
```

```
        Else If choice == 9 Then
            Output "Thank you for using the course planner!"


        Else
            Output choice + " is not a valid option."
        End If
    End While
End Method
```

Operation	Vector	Hash Table	BST
Reading & Parsing lines	O(n)	O(n)	O(n)
Validating Prerequisites	O(n)	O(n)	O(n)
Inserting Courses	O(n)	O(n)	O(n^2)
Total Worst-Case Load Time		O(n^2)	O(n^2)
Average Load Time	O(n)	O(n)	O(n log n)
Memory Usage	Low(O(n))	Moderate (O(n) + empty buckets)	Moderate (O(n) + node pointers)

Worst Case Breakdown

The Vector Load Routine

- Opening and closing the file runs once (Cost = 1).
- Pass 1 reads lines and grabs course IDs (Cost = n).
- Pass 2 creates course objects, checks prerequisites, and appends to the vector (Cost = n).
- Total Worst-Case Cost: The steps run one after another, so the total time is O(n).

Hash Table Load Routine

- Reading the file and checking prerequisites take O(n) time, just like the vector.
- Inserting courses: In the worst case where every course hashes to the exact same bucket, each insert has to walk all the way down a linked list chain. Doing that for every single item takes n* operations.
- Total Worst-Case Cost: O(n) + O(n^2) gives O(n^2). (On average with no collisions, it runs much faster at O(n)).

Binary Search Tree Loding Routine

- Reading the file and checking prerequisites takes O(n) time.
- Inserting courses: In the worst case where the input file is already sorted alphabetically, the tree will not branch out. It turns into a straight line like a linked list. Inserting each new node means comparing it against every node already in the tree(n * n operations).

Advantages and Disadvantages

Vector:

- Pros: It loads the fastest at O(n) and takes up the least memory since it does not need pointers.

- Cons: Searching is slow at O(n) because it has to look through courses one by one. It also does not keep things sorted, so printing in alphabetical order requires a separate sorting algorithm first.

Hash Table:

- Pros: Searching for an individual course is practically instant at O(1) on average.
- Cons: Data is completly unsorted. To print courses alphabetically, everything has to be copied over to a temporary array and sorted, which takes extra memory and time. Long collision chains can also slow down performance.

Binary Search Tree:

- Pros: It keeps courses sorted automatically as they get added. Printing everything in order only needs an in-order traversal at O(n), so no sorting algorithms or temporary arrays are needed. Searching is also fast at O(log n) on average.
- Cons: Each node needs left and right pointers, which uses a bit more memory. it can also degrade to O(n) search time if the input data is pre-sorted.

Recommendation

I'm recommending BTS because:

Advising Workflow: Advisors only load the file once when the app starts. After that, they will run Option 2 (Display Courses) and Option 3 (Final Courses) repeatedly all day.

Easy Alphabetical Printing: Vectors and hash tables make us sort the data every time an advisor wants to vie the course list. A BTS is already sorted, so it can print the list right away.

Fast Searches: Searching a BTS takes around 7 to 8 steps for a 150-course list (O(log n)), which is way faster than checking all 150 courses in a vector (O(n)).

Unsorted File: Because the course file is not pre-sorted, the tree will naturally branch out well and avoid the worst-case slowdown.